Solutions

Exercise 1:

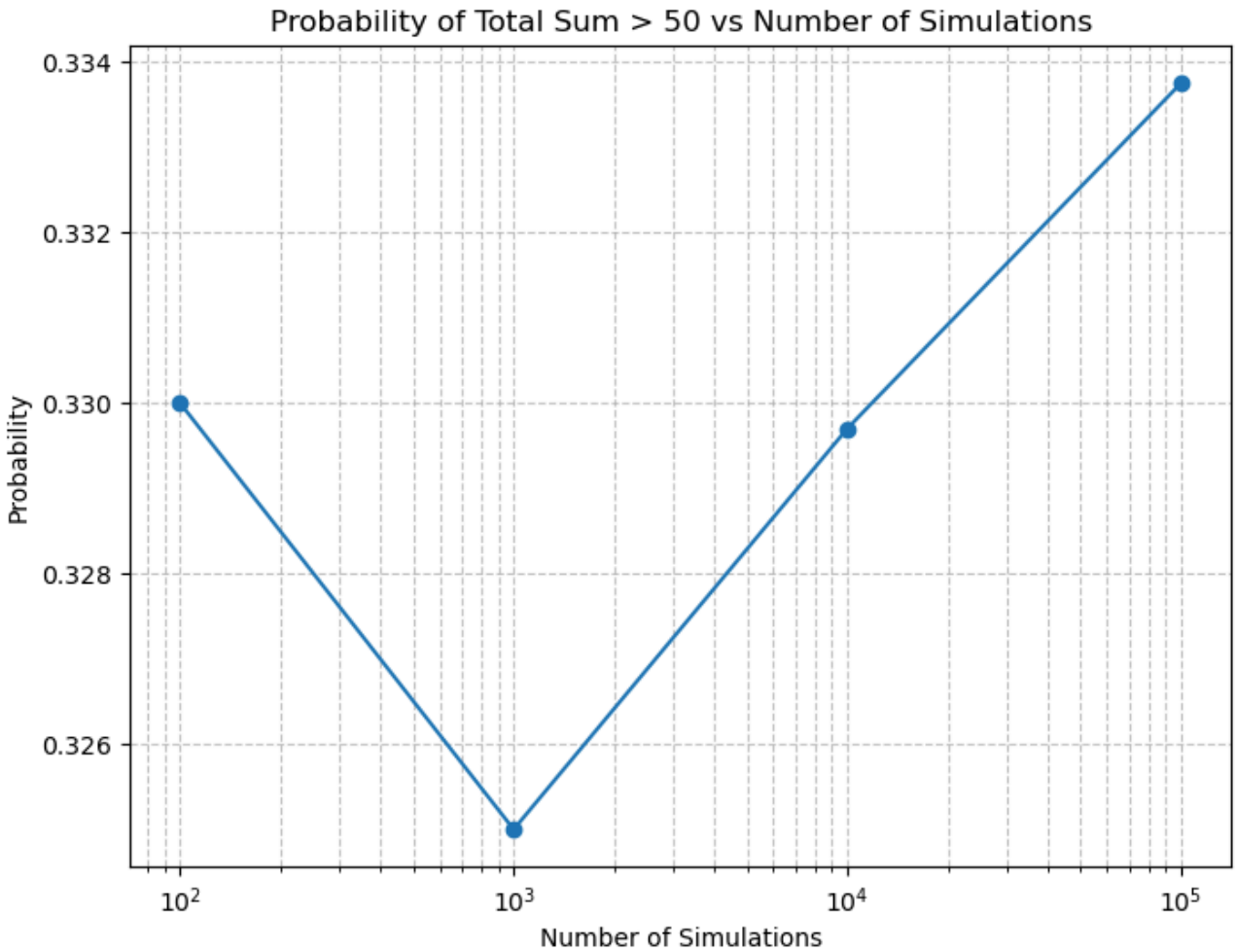
To solve this problem, we'll use Monte Carlo simulation:

1. **Dice Rolling**: We'll simulate rolling the dice for each friend a large number of times to obtain the total sum.
2. **Counting**: We'll count the number of simulations where the total sum is more than 50.
3. **Probability Calculation**: Finally, we'll calculate the probability of getting a total sum of more than 50 by dividing the count of favorable outcomes by the total number of simulations.

Let's proceed to implement this solution using Monte Carlo simulation.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

def simulate_dice_game(num_simulations, num_dice_Alice, num_sides_Alice, num_dice_Bob, n
    # Simulate dice rolls for each friend
    results = []
    for _ in range(num_simulations):
        # Alice's dice roll
        sum_Alice = sum(np.random.randint(1, num_sides_Alice + 1, size=num_dice_Alice))
        # Bob's dice roll
        sum_Bob = sum(np.random.randint(1, num_sides_Bob + 1, size=num_dice_Bob))
        # Charlie's dice roll
        sum_Charlie = sum(np.random.randint(1, num_sides_Charlie + 1, size=num_dice_Char
        # Total sum
        total_sum = sum_Alice + sum_Bob + sum_Charlie
        results.append(total_sum > threshold)
    return np.mean(results)

# Parameters
num_simulations_list = [100, 1000, 10000, 100000]  # Number of Monte Carlo simulations
num_dice_Alice = 4
num_sides_Alice = 4
num_dice_Bob = 6
num_sides_Bob = 6
num_dice_Charlie = 3
num_sides_Charlie = 10
threshold = 50  # Total sum threshold

# Calculate probabilities for different numbers of simulations
probabilities = []
for num_simulations in num_simulations_list:
    probability = simulate_dice_game(num_simulations, num_dice_Alice, num_sides_Alice, n
    probabilities.append(probability)

# Plot probabilities
plt.figure(figsize=(8, 6))
plt.plot(num_simulations_list, probabilities, marker='o', linestyle='-')
plt.xscale('log')
```

```
plt.title('Probability of Total Sum > 50 vs Number of Simulations')
plt.xlabel('Number of Simulations')
plt.ylabel('Probability')
plt.grid(True, which="both", ls="--", alpha=0.7)
plt.show()
```



Exercise 2:

We can use Monte Carlo simulation to estimate the probability of being dealt a poker hand with four cards of the same number when receiving 5 cards from the deck. Here's how we can approach it:

Monte Carlo Simulation: We'll simulate dealing 5 cards from a standard deck of 52 cards many times (let's say 1,000,000 times). Counting: For each simulation, we'll count the number of times a four of a kind hand is dealt. Probability Calculation: Finally, we'll calculate the probability of being dealt a four of a kind hand by dividing the count of favorable outcomes by the total number of simulations.

In [2]:
```
import numpy as np

def simulate_poker_hand(num_simulations):
    # Initialize counter for four of a kind hands
    count_four_of_a_kind = 0

    # Simulate dealing 5 cards many times
```

```
    for _ in range(num_simulations):
        # Create a deck of cards
        deck = np.arange(1, 53)
        np.random.shuffle(deck)

        # Deal 5 cards
        hand = deck[:5]

        # Check for four of a kind
        values, counts = np.unique(hand % 13, return_counts=True) # Check for repeated c
        if np.max(counts) >= 4:  # If any card appears four times or more
            count_four_of_a_kind += 1

    # Calculate probability of four of a kind
    probability_four_of_a_kind = count_four_of_a_kind / num_simulations
    return probability_four_of_a_kind

# Number of Monte Carlo simulations
num_simulations = 1000000

# Perform Monte Carlo simulation
probability = simulate_poker_hand(num_simulations)
print("Estimated probability of being dealt a four of a kind hand:", probability)
```

```
Estimated probability of being dealt a four of a kind hand: 0.000243
```

In [ ]: